



SAKINAH

Complete Application Blueprint

AS BUILT • SECURED

Version 1.0 | 07 September 2026

Verified from the secured repository. Secret values excluded.

Contents

- Product Requirements Document
- Technical Requirements Document
- App / Web Flow Document
- UI / UX Design Document
- Backend Schema and API Document
- Implementation Plan
- Traceability appendix

Product Requirements Document

As Built

Product: Sakinah - Islamic Companion

Document status: As Built with implementation-status labels

Version: 1.0 | Date: 2026-09-07

1. Product title and pitch

Sakinah is an installable, mobile-first Islamic companion that brings Quran study, Hadith, prayer support, remembrance, and daily guidance into one calm web application.

2. Executive summary

Sakinah serves guests and registered Muslims on Android, iOS, and desktop browsers. Guests can use most content locally; registered users can sync supported bookmarks and Tasbeeh records through an account. The product is delivered as a Progressive Web App (PWA), with an install gate in browsers and first-run onboarding in standalone mode.

3. Purpose and problem

Islamic resources are often distributed across unrelated websites and apps. Sakinah reduces switching by placing worship utilities, source content, personal progress, and shareable reminders under a consistent interface. It addresses discoverability, mobile convenience, location-aware prayer support, and continuity across sessions.

4. Target users and stakeholders

- Muslims seeking a daily Quran, Hadith, prayer, Dua, and Dhikr companion.
- Learners who want topical Quran references, recitation, and structured lessons.
- Travellers needing local prayer time and Qibla assistance.
- Guests who prefer no account.
- Registered users who need supported cloud synchronization.
- Product owner, content reviewers, support contact, Supabase/Vercel operators.

Roles

Role	Capabilities
Guest	Browse content; use utilities; retain selected data locally in the browser
Authenticated user	Guest capabilities plus supported Supabase synchronization
Administrator	Not implemented or verified from repository

5. Pain points addressed

- Multiple disconnected religious tools.
- Loss of progress between visits.
- Difficulty finding Quran passages by topic or location.

- Static prayer schedules that do not follow current coordinates/date.
- Poor mobile readability and lack of installable access.

6. Current scope and modules

Module	As-built capability	Status
Home	Live prayer banner, search handoff, Ayat card, Flashes, quick actions, Hajj/Umrah entry	Implemented
Quran	Surah/Juz browsing, Arabic/Urdu/English editions, recitation, bookmarks/favorites, 15/16-line visual modes, subjects, 38 lessons	Implemented with provider dependency
Seerat-un-Nabi	Seven seasons and episode launcher using one YouTube playlist	Partially implemented/content-dependent
Hadith	Eight collections, collection → topic → Hadith flow, language controls where provider data exists, share/save	Implemented with translation/provider limitations
Prayer	Geolocation, current-date AlAdhan calculation, next-prayer countdown, reminder controls	Implemented; background reliability platform-dependent
Qibla	Coordinate bearing plus device orientation	Implemented; sensor accuracy platform-dependent
Tasbeeh	Presets, vibration, target progress, local history, custom Tasbeeh and optional user sync	Implemented
Books	Categorized visual library and reader-oriented entries	Partially implemented; complete licensed book corpus/download verification absent
Duas	Local Hisnul Muslim dataset and fallback content	Implemented; corpus completeness not independently verified
Ibadat	Salah, fasting, Hajj, Umrah, Janaza guidance cards	Implemented as concise guidance
More	Flashes, Duas, Salah, Qibla, 99 Names, Shahadat, Waqf, Tasbeeh, Books	Implemented
Account	Email/password, reset, Google OAuth, session persistence, profile summary	Implemented; remote provider settings not repository-verifiable
PWA	Manifest, install UI, offline fallback, versioned updates	Implemented

7. Out of scope

- Native App Store/Play Store binaries.
- Payments, subscriptions, commerce, or advertising.
- Admin console or editorial CMS.
- Social network, public profiles, or user-to-user messaging.
- Scholarly certification of third-party religious content.
- Guaranteed background Azan execution when the OS suspends a PWA.
- Server-owned push campaign scheduler.
- Dedicated book-file storage/download service.

8. Functional requirements

- PRD-FR-01: The browser view shall present installation guidance; standalone first run shall present onboarding.
- PRD-FR-02: Users shall navigate Home, Quran, Hadith, Ibadat, and More from the bottom bar.
- PRD-FR-03: Every primary view shall retain access to the header menu and persistent prayer banner.
- PRD-FR-04: The Quran experience shall support Surah/Juz selection, translations, recitation, reading preferences, favorites, and last-read markers.
- PRD-FR-05: Hadith navigation shall proceed collection → chapter/topic → Hadith.
- PRD-FR-06: Prayer times and Qibla shall use current permitted coordinates.
- PRD-FR-07: Tasbeeh shall count taps, support reset/history, and allow bounded custom records.
- PRD-FR-08: Guests shall be usable without account creation.
- PRD-FR-09: Authenticated ownership data shall be limited to its owner.
- PRD-FR-10: The application shall expose light/dark preference and install/update behavior.

9. Non-functional requirements

- Mobile-first presentation within a 560 px application shell on wider displays.
- Installable standalone PWA over HTTPS.
- Current application code fetched reliably with offline fallback.
- Arabic-capable typography and readable translation layouts.
- Security headers, strict input bounds, RLS ownership, and no frontend secret keys.
- Graceful loading, empty, error, and permission-denied states where implemented.
- No guarantee of offline availability for every third-party corpus item.

10. User stories

- As a guest, I want to explore without registering, so that I can evaluate the companion privately.
- As a reader, I want Urdu or English Quran translation, so that I can understand an Ayah.
- As a learner, I want a lesson to open its exact Ayah, so that study remains contextual.
- As a worshipper, I want local prayer times, so that the schedule fits my location.
- As a traveller, I want a Qibla bearing, so that I can orient prayer.
- As a registered user, I want saved bookmarks, so that supported progress follows my account.
- As a Dhikr user, I want a custom Tasbeeh, so that I can use my own title, meaning, target, and image.
- As a mobile user, I want Home Screen installation, so that Sakinah launches like an app.

11. Business rules

- Guest data is local and can be lost if browser/app storage is cleared.
- Cloud rows must be owned by the authenticated Supabase user.
- Custom images are JPG/PNG/WebP and no larger than 1.5 MB.
- Custom targets are integers from 1 to 100,000.
- Prayer calculation uses AIAdhan method 2 and school 0 for all users.
- Fiqh-specific filtering is not present.
- Third-party content remains subject to provider availability and licensing.

12. Assumptions and constraints

- Users grant geolocation, notification, and orientation permission when needed.

- Network access is needed for uncached provider content.
- iOS installation is initiated through Safari Share → Add to Home Screen.
- Browser/OS policies may prevent exact background reminder delivery.
- Supabase and OAuth Dashboard configuration is managed outside this repository.

13. Success criteria

- Production build completes without error.
- Security regression suite remains 100% passing.
- All 114 Surahs and Juz 1–30 can be requested from the provider within validated ranges.
- A permitted location yields current-date prayer data or a clear recoverable error.
- Guest launch does not require account creation.
- Authenticated cross-user database access is denied by RLS in an owner-run two-account test.
- PWA manifest and service worker are served with correct types/cache policy.

14. Known limitations

- No repository-owned backend or distributed notification scheduler.
- Complete book downloads are not verified.
- Some Hadith Urdu/Arabic availability depends on matching remote editions.
- “15-line/16-line” is a CSS reading mode over provider pages, not verified photographic Indopak pagination.
- Seerah uses playlist indexing rather than a repository-managed episode catalog.
- Account deletion/export is absent.

15. Glossary

Term	Meaning
Ayah	A Quran verse
Surah	A Quran chapter
Juz/Para	One of 30 Quran divisions
Hadith	Narration about the Prophet's teachings/practice
Dhikr/Tasbeeh	Remembrance and counting practice
Qibla	Direction of the Kaaba for prayer
PWA	Installable web application with manifest/service worker
RLS	Database Row-Level Security

Technical Requirements Document

As Built

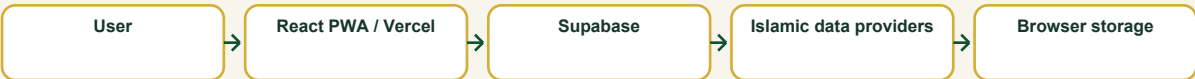
Product: Sakinah - Islamic Companion | Status: As Built | Version: 1.0 | Date: 2026-09-07

STACK LOCKED: Future implementation must not substitute frameworks, database, ORM, authentication model or hosting architecture unless the owner approves the change and this TRD is updated first.

1. System overview and architecture

Sakinah is a static single-page React PWA. Vercel serves Vite output; the browser communicates directly with Supabase and read-only Islamic content providers. No repository-owned Node/Express API runs in production.

Verified flow diagram



Architecture style: client-side SPA/PWA with Backend-as-a-Service.

2. Repository structure

Path	Purpose
src/App.jsx	Composition, navigation, core Quran/Hadith/prayer/Tasbeeh/profile views
src/components/Upgrade.jsx	Home, onboarding, share cards, Duas, Ibadat, More
src/components/QuranExtras.jsx	Seerah, recitation, subjects, Fahm lessons, line modes
src/components/AuthModal.jsx	Email/password/reset/Google auth UI
src/lib/islamicApi.js	External content API adapter and validation
src/lib/supabase.js	Supabase client/session configuration
src/lib/security.js	Security validation helpers
src/*.css	Component and design-system styling
public/	Manifest, service worker, offline page, icons, Dua JSON
supabase/schema.sql	Tables, trigger, and RLS policies
tests/security.test.js	Node-native security regression tests
vercel.json	SPA rewrite, CSP, security/cache headers

3. Locked stack and versions

Concern	Requirement
Runtime/build	Node.js compatible with Vite 5; exact production Node not repository-pinned
UI	React 18.3.1, React DOM 18.3.1
Build	Vite 5.4.21; React plugin 4.7.0 installed
Icons	lucide-react 0.468.0
CSS	Tailwind 3.4.19, PostCSS 8.5.27, Autoprefixer 10.5.4, custom CSS
Auth/data SDK	@supabase/supabase-js 2.57.0
Package manager	npm; package-lock.json
Database	Supabase PostgreSQL; no ORM
Hosting	Vercel static deployment

State management uses React hooks plus local/session storage. Routing is an allow-listed view map using the view query parameter; no React Router.

4. Authentication and roles

Supabase Auth uses email/password, password-reset email, and Google OAuth. Client settings enable PKCE, URL session detection, token refresh, and persistent storage key `sakinah-auth-session`. Guest and authenticated-user states exist; no admin role exists. Authorization is database-enforced with `auth.uid()` RLS.

5. Integrations

Service	Purpose	Data sent
Supabase	Auth, profiles, bookmarks, Tasbeeh sessions	Credentials/tokens and owner data
Al Quran Cloud	Surah/Juz/edition text and audio metadata	Validated content identifiers
AlAdhan	Prayer times and Asma-ul-Husna	Coordinates/date for prayer requests
Hadith API via jsDelivr	Collection/section JSON	Validated collection paths
YouTube nocookie	Seerah playlist embeds	Standard embed/browser metadata
Unsplash	Flash/Hajj images and card composition	Image request metadata
Google Fonts	Amiri, DM Sans, Playfair Display	Font request metadata
WhatsApp	Support deep link	Only when user activates link

6. Storage

- Local storage: UI preferences, guest bookmarks/history, custom Tasbeeh, notification settings/log, Supabase session.
- Session storage: browser install-gate dismissal.
- Cache Storage: versioned shell/code/images and Quran responses; prayer-coordinate requests are excluded.
- Supabase tables: `profiles`, `bookmarks`, `tasbeeh_sessions`.
- Dedicated file/object storage: not present.

7. Environment variables

Name	Exposure	Purpose
VITE_SUPABASE_URL	Public browser bundle	Supabase project origin
VITE_SUPABASE_PUBLISHABLE_KEY	Public browser bundle	Supabase publishable/anon access

Never use VITE_ for service-role, database, OAuth client-secret, SMTP, webhook, or private keys. Missing/placeholder values leave account features unavailable while preserving guest mode.

8. Development, build, tests, deployment

```
npm install
npm run dev
npm run test:security
npm run build
npm run preview
npm audit --omit=dev
```

dev and preview bind to 127.0.0.1. Build output is dist/. Vercel rewrites SPA requests to index.html; real files take precedence. No Docker configuration, migration runner, seed command, lint script, or type-check script exists.

9. API conventions

- External fetch adapters throw short generic errors.
- Numeric identifiers are bounded; dynamic path segments are allow-listed by syntax.
- UI loading/error/empty state is managed per component.
- Supabase queries use SDK filters and RLS; no raw SQL from the browser.
- No repository-owned JSON API response convention exists.

10. Logging and error handling

The root error boundary prevents a blank screen. Auth logs contain only operation mode/redacted codes; identifiers and raw provider errors are excluded. Provider-facing user messages are short and recoverable. No centralized telemetry or correlation IDs exist.

11. Security controls

- Vercel CSP, HSTS, anti-framing, MIME, referrer, permissions, opener/resource headers.
- React escaped rendering; no dangerous HTML sink found.
- OAuth destination restricted to configured HTTPS Supabase origin.
- Auth and content inputs bounded.
- Custom image allow-list and 1.5 MB limit.
- Supabase RLS ownership.
- Production dependency audit clean.
- See docs/security/SECURITY-AUDIT.md for findings and owner actions.

12. Performance/scalability

Lazy-loaded Quran extras reduce initial work. Long provider corpora are fetched on demand; some lists are locally paginated. Static Vercel assets scale at CDN level. Direct third-party calls avoid a server bottleneck but inherit provider rate/availability limits. Supabase payload growth and JSON bookmark size require future quotas/index review.

13. Naming and conventions

- React components: PascalCase functions.
- Hooks/helpers: camelCase.
- CSS: feature-oriented kebab-case selectors.
- View identifiers: lower-case strings in the central map.
- Supabase tables/columns: snake_case.
- Security changes require regression tests under tests/.

14. Hard constraints / DO NOT USE or change

- Do not replace React/Vite, Supabase, PostgreSQL, Vercel, or the SDK auth model without owner approval.
- Do not add a second router/state framework casually.
- Do not put secrets in VITE_* or source.
- Do not bypass RLS or use a service-role key in the browser.
- Do not cache coordinate-bearing prayer URLs.
- Do not weaken CSP to broad * sources.
- Do not alter the three existing table semantics without migration/data review.
- Do not claim reliable background push/Azan without a server push architecture and platform tests.

15. Technical debt and unresolved risk

- Vite/esbuild development advisories need an owner-approved Node/Vite major upgrade.
- No browser E2E, lint, type-check, or accessibility automation.
- Supabase remote configuration is not infrastructure-as-code.
- Book completeness/licensing and some translations are not verified.
- Very large provider data may need virtualization.
- Feedback is local-only; account deletion/export is absent.
- Service-worker data expiration is version-based rather than TTL-based.

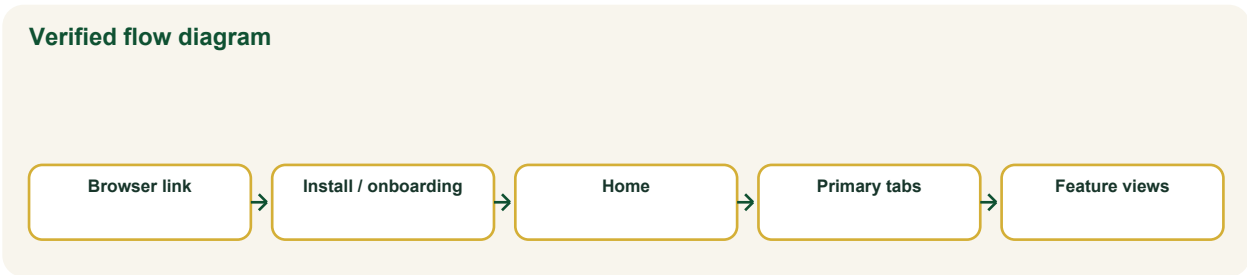
App / Web Flow Document

As Built

Product: Sakinah | Status: As Built | Version: 1.0 | Date: 2026-09-07

1. Roles and global navigation

Guests and authenticated users share the same content screens. Account-bound save operations require authentication. No administrator/privileged screen exists. Global navigation consists of the sticky header/hamburger, persistent prayer widget, bottom bar (Home, Quran, Hadith, Ibadat, More), and back-to-Home behavior on secondary views.



2. Screen inventory

Screen / view	Purpose	Access	Main data	Key actions	Next destinations
Install gate	PWA installation guidance	Public browser	Platform/install event	Install, iOS instructions, continue web	Standalone app or Home
Onboarding	Feature introduction	First standalone launch	Static slides	Next, All Done	Home
Home (home)	Daily overview	All	Prayer state, rotating Ayat, Flashes	Search handoff, share, quick access	Quran, Duas, Tasbeeh, Ibadat, Flashes
Quran (quran)	Quran hub	All	Remote Quran and local progress	Choose mode/reader, favorite, bookmark, recite	Reader, favorites, lessons, subjects
Al-Quran reader	Surah/Juz reading	All	Arabic/English/Urdu/ audio	Search, select, translate, customize, save	Quran hub
Seerat-un-Nabi	Video series	All	Static season/episode metadata	Select season/episode, play	Quran hub
15/16-Line	Page-oriented reading modes	All	Quran page API	Previous/next page	Quran hub
Recitation	Audio selection	All	Surah API/audio	Select and play	Quran hub
Subjects	Topical Quran index	All	Static 36-topic map	Search, open Ayah reference	Al-Quran
Fahm-ul-Quran	38 lessons	All	Lesson refs + Quran API	Expand, recite, open Ayah	Al-Quran
Hadith (hadith)	Collection selection	All	Eight remote collections	Select book	Topic list
Hadith topic list	Chapters within collection	All	Remote section metadata	Language preference, choose topic	Hadith list
Hadith list	Narrations	All; saving requires account for cloud	Remote editions	Share, favorite	Topic list
Ibadat (ibadat)	Worship guides	All	Static bilingual guidance	Select Salah/Sawm/Hajj/Umrah/Janaza	Detail cards
More (more)	Secondary feature launcher	All	Static menu	Open feature	Feature view
Prayer (prayer)	Daily prayer schedule	All	Geolocation + AIAdhan	Refresh location, toggle prayers, request notifications	Home
Qibla (qibla)	Direction finder	All	Coordinates/sensor	Enable compass	Home
Tasbeeh (tasbeeh)	Dhikr counter	All	Local/cloud records	Count, undo, reset, create/edit	Home/Profile
Books (library)	Islamic book catalog	All	Repository metadata/content	Search/filter/open	Reader entry
Calendar (calendar)	Gregorian/Hijri context	All	Device date/static events	Month navigation	Home
Flashes (flashes)	Shareable cards	All	Static images/text	Share rendered image	Home

Screen / view	Purpose	Access	Main data	Key actions	Next destinations
Duas (duas)	Supplication browser	All	Local JSON/fallback	Search/browse	Home
Shahadat (shahadat)	Declaration and meaning	All	Static content	Read	Home
99 Names (names)	Asma-ul-Husna	All	AIAdhan/local cache	Browse	Home
Waqf (waqf)	Stop-sign guide	All	Static guide	Browse	Home
Profile (profile)	Account and saved summary	Auth-aware	Supabase user/local stats	Sign in/out	Auth/Home
Sidebar	Account/settings/support/FAQ	All	Preferences/log	Toggle, open account/support	Profile/features
Auth modal	Login/signup/reset/OAuth	Public	User input/Supabase	Submit, Google, switch mode	Profile/current view
Offline fallback	Network recovery	Public/offline	Cached shell	Retry	Home

3. Authentication flows

Verified flow diagram



- Login: validated email/password → Supabase → session closes modal.
- Registration: validated name/email/password → Supabase; session or confirmation instruction depending external Auth setting.
- Reset: enumeration-resistant message regardless of account existence.
- Logout: `supabase.auth.signOut()`; UI returns to guest state.
- Expired session: Supabase auto-refreshes when possible; otherwise auth-state listener returns guest state.
- Access denied: RLS rejects non-owner data; UI-specific error quality varies by operation.

4. Core workflows

Quran

Mode → choose Juz/Surah/edition or extra section → loading → content → optional translation/audio → local save; authenticated save additionally writes through Supabase. Subject/lesson references set a target and return to AI-Quran.

Hadith

Collection → provider collection metadata → real section/topic → remote section data → language display where present → share/favorite. A missing edition yields the available language or an error state.

Prayer/Qibla

Screen/app load → permission request → coordinates → validated request/current date → AIAdhan times → countdown and reminder settings. Qibla derives bearing locally and combines it with device orientation when permission/sensor exists.

Tasbeeh

Select preset/custom → tap/undo → save and reset → local rolling history; authenticated sessions also insert a user-owned cloud row. Custom flow validates title, meaning, target, and image before local/cloud persistence.

5. Data states by screen type

Screen type	Loading	Empty	Error	Populated
Quran/Hadith	Loading note/skeleton where implemented	Empty component/list	Provider-specific message	Reader/list/cards
Prayer/Qibla	Location required / countdown placeholder	No coordinates	Permission/service message	Times/bearing
Profile/history	Auth/session readiness	Synced history placeholder	Some queries log rather than show error	User/stats
Duas/Names	Loading message	Empty list	Reconnect note	Searchable cards/grid
Flashes	Animated skeleton	Not expected	Gradient fallback	Decoded background/card content
Books/Ibadat/static	Immediate	Not expected	Limited error surface	Static cards

6. Search/filter/import/export

- Home search routes to Quran but does not preserve/query the term: partial implementation.
- Quran and Subjects search filter local/remote-loaded metadata.
- Books, Duas, and Hadith expose module-specific filtering where coded.
- Share uses Web Share and file sharing when supported; fallback downloads/copies.
- No bulk export/import exists.
- Custom image selection is the only file import; no server file endpoint exists.

7. Error, recovery, and destructive paths

- Root render failures show a retry screen through `AppErrorBoundary`.
- Offline navigation falls back to `public/offline.html`.
- Provider failures show local messages; cached Quran/app resources may remain usable.
- Permission denial retains the app and explains that location/notification is required.
- Unknown view values fall back to Home; no dedicated 404 screen.
- Bookmark removal and sign-out are immediate; no confirmation dialog.
- No account deletion or destructive database administration flow exists.

8. Responsive/mobile rules

The app is capped at 560 px and centered on larger displays. Bottom navigation is fixed with safe-area padding. The sidebar slides from the right. Browser-only view presents the install gate; standalone mode gates first-run onboarding. iOS uses manual Add to Home Screen instructions because `beforeinstallprompt` is unavailable.

9. Missing or unverified flows

- Admin workflows: not implemented.
- Account deletion/data export: not implemented.
- Server-scheduled push: not implemented.

- Reliable auto-Azan in suspended state: not verified.
- Complete offline Quran/Hadith/books: not verified.
- Cross-user RLS behavior must be owner-tested against the remote project.

UI / UX Design Document

As Built

Product: Sakinah | Status: As Built | Version: 1.0 | Date: 2026-09-07

1. Brand identity and assets

Sakinah uses a calm, devotional visual language: deep emerald, warm gold, cream surfaces, rounded cards, editorial serif headings, and Arabic-friendly reading text. Approved repository assets are `public/assets/sakinah-logo.png`, `public/icons/icon-192.png`, and `public/icons/icon-512.png`.

2. Verified palette

Token/use	Hex	RGB
Primary emerald --green	#0F5132	15, 81, 50
Secondary emerald --green2	#176B49	23, 107, 73
Gold --gold	#D4AF37	212, 175, 55
Cream --cream	#F8F5ED	248, 245, 237
Card --card	#FFFDFA	255, 253, 250
Text --text	#17352A	23, 53, 42
Muted --muted	#718078	113, 128, 120
Line --line	#E5E7DF	229, 231, 223
Page background	#E7EBE8	231, 235, 232
Dark cream/card	#101A16 / #17241E	16,26,22 / 23,36,30

Dark mode remaps cream, card, text, muted, line, shadow, and active accents through `html[data-theme=dark]`.

3. Typography

Role	Family	Typical values
Interface/body	DM Sans	8–13 px in compact mobile UI; 400–700
Display/headings	Playfair Display	17–30 px; 600–700
Quran/Arabic	Noto Naskh Arabic (component declarations) and Amiri import	Reader commonly 23–30 px; line-height 1.65–2
Quote fallback	Georgia	Decorative quote glyph

Google Fonts imports Amiri, DM Sans, and Playfair Display. Some components reference Noto Naskh Arabic without a matching Google import, so actual rendering may fall back to an installed Arabic font: inconsistency.

4. Spacing, radius, shadows

No formal numeric token scale exists. Repeated spacing clusters are 4, 8–12, 14–20, 22–30 px. Radius values vary intentionally: 8–14 px controls, 13–20 px cards, 22–30 px hero/modal surfaces, and 50% circles. Primary shadow is `0 12px 35px rgba(17,65,44,.09)`; deeper overlays use approximately `0 25–30px 80–90px rgba(0,0,0,.3–.4)`.

5. Layout and breakpoints

- `.app`: full width, maximum 560 px, minimum viewport height, centered.
- Main content: 20 px horizontal padding.
- Primary grids: two columns; Quran Juz grid uses three columns.
- Primary breakpoint: `min-width: 561px` adds 24 px page inset and rounded application shell.
- Mobile sidebar rule: `max-width: 560px` anchors drawer to the right.
- Safe-area bottom padding is used in bottom navigation.

6. Navigation patterns

- Sticky 76 px header with brand/back button, centered secondary title, and hamburger.
- Fixed 76 px bottom navigation: Home, Quran, Hadith, Ibadat, More.
- Right slide-out account/settings drawer with backdrop.
- Persistent prayer banner at the top of every main screen.
- Secondary screens use a Home-return back action rather than browser history traversal.

7. Components and states

Component	Appearance/states	Source
Header	Brand vs back state; menu circle	<code>src/App.jsx, styles.css</code>
Prayer banner	Emerald gradient, mosque/crescent animation, countdown	<code>Upgrade.jsx, upgrade.css</code>
Bottom navigation	Muted/default and emerald/active	<code>App.jsx, styles.css</code>
Feature cards	White/cream, border, icon tile, chevron	<code>App.jsx, styles.css</code>
Buttons	Primary emerald, gold update, outlined/ghost, disabled opacity	Multiple CSS files
Inputs	Rounded card/background; search leading icon; native selects	<code>styles.css, auth.css, content.css</code>
Cards	13–25 px radius, line border, occasional shadow	All view CSS
Modal	Centered 26 px auth dialog on translucent backdrop	<code>AuthModal.jsx, auth.css</code>
Drawer	Right fixed sidebar, shade, grouped settings	<code>App.jsx, upgrade.css</code>
Toast/note	Pale green compact status or centered muted note	<code>styles.css</code>
Badges	Small uppercase gold labels; next-prayer chip	Multiple
Loading	Suspense text, notes, Flash shimmer skeleton	<code>App.jsx, upgrade.css</code>
Empty	Centered moon icon and muted message	<code>App.jsx</code>
Error	Root retry screen or inline note/message	<code>main.jsx, views</code>
Icons	Lucide React line icons; logo PNG	Components/public

Tables are not a meaningful app UI pattern. There is no general toast manager. Validation messages are inline, especially auth and custom-image errors.

8. Motion

- Onboarding/splash uses layered fixed surfaces and transitions.
- Mosque floats in 3D perspective; crescent glows.
- Counter scales on active press.
- Flash skeleton shimmers until image decode.
- Reduced-motion media query disables selected animations, not every transition.

9. Forms and validation UX

Auth uses required fields, email type, length limits, password autocomplete, busy state, and role=status feedback. Custom Tasbeeh uses required title/target, bounded number input, MIME-restricted file picker, and inline image error. Some provider/action failures remain console-only rather than surfaced near the initiating control.

10. Accessibility review

Area	As-built assessment
Contrast	Core emerald/cream combinations are generally strong; very small muted text needs device review
Keyboard	Native buttons/inputs are keyboard-operable; focus styling is inconsistent/not systematically defined
Labels	Many icon buttons have labels, but some image/control labels are incomplete
Semantics	Auth modal has dialog semantics; sections/headings used; drawer lacks explicit dialog/navigation role
Text selection	Globally disabled except inputs/textareas, reducing native reading/copy accessibility
Font size	Numerous 8–10 px labels are visually compact and may fail user readability expectations
Motion	Partial reduced-motion support exists
Arabic	<code>dir=rtl</code> used in selected components; not systematic across all Arabic content

11. Component inventory

- `src/App.jsx`: Header, Sidebar, Quran reader/control/favorites/edit browser, Tasbeeh, Prayer, Qibla, Library, Hadith, Calendar, Profile, InstallGate, scheduling and empty state.
- `src/components/Upgrade.jsx`: EnhancedOnboarding, PersistentPrayer, EnhancedHome, FlashCard, Flashes, Duas, Shahadat, Ibadat, More, Waqf, Names.
- `src/components/QuranExtras.jsx`: SeerahSeries, RecitationLibrary, QuranSubjects, FahmCourse, LineMushaf.
- `src/components/AuthModal.jsx`: accessible account dialog and flows.

12. Current inconsistencies

- Noto Naskh Arabic is referenced but not imported alongside the three declared Google fonts.
- Font sizes are often below 11 px.
- Focus-visible treatment is not standardized.
- Both legacy and enhanced home/onboarding components remain in source; enhanced versions are active.
- Some CSS is minified into long lines while other files are expanded.

- Status/error presentation varies by module.
- Global no-selection behavior conflicts with copying Quran/Hadith text and some accessibility expectations.

13. Hard aesthetic rules

- Preserve emerald #0F5132 and gold #D4AF37 as primary identity.
- Preserve cream/card warmth and restrained dark mode.
- Keep mobile-first, centered 560 px shell and safe-area bottom bar.
- Use Playfair for editorial headings and an actual Arabic-capable font for scripture.
- Keep religious content uncluttered and never cover Arabic text with decoration.
- Use rounded, calm surfaces; avoid neon colors, dense gradients, or unrelated illustration styles.
- Respect reduced motion and improve focus/readability before adding further animation.

Backend Schema and API Document

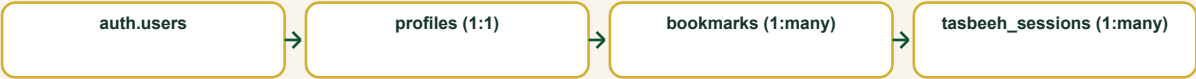
As Built

Product: Sakinah | Status: As Built | Version: 1.0 | Date: 2026-09-07

1. Backend architecture

There is no repository-owned HTTP backend, controller, server runtime, ORM, Prisma schema, or server route. The browser uses Supabase Auth/PostgREST directly. Database DDL is in `supabase/schema.sql`; external Islamic content uses browser fetch adapters in `src/lib/islamicApi.js`.

Verified flow diagram



2. Database inventory

public.profiles

Field	Type	Required	Default	Constraints
id	uuid	Yes	none	PK; FK → auth.users(id); cascade delete
name	text	Yes	empty string	none beyond not-null
email	text	No	null	none
city	text	No	null	none
preferences	jsonb	Yes	{}	not-null
created_at	timestampz	Yes	now()	not-null
updated_at	timestampz	Yes	now()	not-null; no auto-update trigger

public.bookmarks

Field	Type	Required	Default	Constraints
id	bigint identity	Yes	generated	PK
user_id	uuid	Yes	none	FK → auth.users(id); cascade delete
kind	text	Yes	none	one of ayah, hadith, book
reference	text	Yes	none	unique with user/kind
payload	jsonb	Yes	{}	not-null; shape/size not constrained
created_at	timestamptz	Yes	now()	not-null

Unique index/constraint: (user_id, kind, reference).

public.tasbeeh_sessions

Field	Type	Required	Default	Constraints
id	bigint identity	Yes	generated	PK
user_id	uuid	Yes	none	FK → auth.users(id); cascade delete
dhikr	text	Yes	none	not-null
count	integer	Yes	none	count > 0
created_at	timestamptz	Yes	now()	not-null

3. Relationships, ownership, lifecycle

Every row belongs to one Supabase Auth user. Profile is one-to-one; bookmarks and sessions are one-to-many. Deleting an Auth user cascades to all three tables. There is no soft delete, audit table, record version, or application-defined retention schedule.

RLS is enabled for each table. FOR ALL policies require auth.uid() to equal id or user_id in both USING and WITH CHECK. This governs select, insert, update, and delete. No role enum or admin bypass is defined in repository SQL.

A security-definer trigger handle_new_user() inserts profile ID/name/email after an Auth user is created and fixes search_path=public. Existing profile conflicts are ignored.

4. Enums, indexes, seed/migrations

- No PostgreSQL enum type exists; bookmarks.kind uses a check constraint.
- Explicit index beyond primary/unique/FK backing behavior: none declared.
- There is one idempotent-oriented SQL setup file, not an ordered migration system.
- No seed script exists.
- Re-running the file may fail at create policy because policies are not conditionally dropped/created.

5. Supabase operation inventory

These are SDK operations, not custom HTTP endpoints.

Operation	Method/path concept	Auth	Ownership	Request/response
Get session	Supabase Auth <code>getSession</code>	Optional	Current browser session	Session/user or null
Auth-state listener	Supabase Auth channel	Optional	Current session	Auth event/session
Sign up	Auth <code>signUp</code>	Public/rate-limited externally	New account	Email/password/name → user/session/error
Sign in	Auth <code>signInWithPassword</code>	Public	Account credentials	session/user/error
Google OAuth	Auth <code>signInWithOAuth</code>	Public	Provider identity	HTTPS authorize URL
Password reset	Auth <code>resetPasswordForEmail</code>	Public	Email purpose	generic client message
Sign out	Auth <code>signOut</code>	Authenticated	Current session	completion/error
Load Ayah saves	<code>bookmarks.select</code> filtered <code>kind=ayah</code>	Authenticated	RLS	<code>reference,payload[]</code>
Upsert/delete Ayah	<code>bookmarks.upsert/delete</code>	Authenticated	RLS + supplied <code>user_id</code> on insert	row/error
Load custom Tasbeeh	<code>bookmarks.select</code> , <code>kind book</code> , <code>reference like tasbeeh:%</code>	Authenticated	RLS	payload list
Save custom Tasbeeh	<code>bookmarks.upsert</code>	Authenticated	RLS	bounded JSON payload
Save Tasbeeh session	<code>tasbeeh_sessions.insert</code>	Authenticated	RLS	positive count row/error
Save/delete Hadith	<code>bookmarks.upsert/delete</code> , <code>kind hadith</code>	Authenticated	RLS	narration payload/error

No REST status/error wrapper is controlled by this repository; Supabase SDK/PostgREST semantics apply.

6. External read API inventory

Method	Endpoint pattern	Purpose	Auth	Validated inputs	Main failure
GET	<code>api.alquran.cloud/v1/surah</code>	Surah metadata	None	none	generic unavailable error
GET	<code>/surah/{1..114}/editions/...</code>	Text/translations/audio	None	bounded Surah	load error
GET	<code>/edition</code>	Edition list	None	response filtered	unavailable
GET	<code>/juz/{1..30}/{edition}</code>	Juz content	None	number + edition syntax	load error
GET	<code>api.aladhan.com/v1/timings/{date}</code>	Prayer times	None	lat/lon/date bounds; fixed method/school	unavailable
GET	<code>api.aladhan.com/v1/asmaAlHusna</code>	99 Names	None	none	inline reconnect error
GET	<code>cdn.jsdelivr.net/.../{path}.json</code>	Hadith collection/section/item	None	lower-case safe path syntax	unavailable
GET/embed	<code>youtube-nocookie.com/embed/video series</code>	Seerah video	None	repository-fixed playlist/index	embed failure
GET	<code>/assets/hisnul-muslim.json</code>	Dua corpus	Same origin	none	fallback list

There are no upload, webhook, payment, admin, export, or server-side URL-fetch endpoints.

7. Validation and security controls

- Auth email format/254 max, name 80 max, password 8–128 on signup.
- Surah 1–114, Juz 1–30, edition syntax up to 64 characters.
- Coordinates latitude –90..90 and longitude –180..180.
- Hadith path syntax prevents traversal/operator injection.
- Custom Tasbeeh title 80, meaning 240, target 1–100,000.
- Image JPEG/PNG/WebP, ≤1.5 MB, verified data-URL prefix.
- React escapes rendered data; no raw HTML sink found.
- RLS prevents IDOR/BOLA across declared tables when deployed as written.
- OAuth provider navigation is exact-origin/HTTPS constrained.

8. Search/pagination/sort conventions

- Quran/subject/book/Dua search is client-side after metadata/content load.
- Hadith provider sections drive topic navigation.
- Some lists expose local “load more”/pagination controls; no universal server pagination contract exists.
- No arbitrary sort-field input reaches SQL.

9. Error behavior

Provider adapters throw generic `Error` messages. Auth UI maps raw provider errors to generic public messages and logs redacted codes. Supabase data-operation error handling is inconsistent: Quran loading logs an error while several writes do not display a failure. Correlation/request IDs are not implemented because there is no application server.

10. Upload/payment/rate limits

Upload is limited to a custom image encoded in JSON, not a dedicated file endpoint. Server-side magic-byte checking, malware scan, and quota are absent. Payments/webhooks are not applicable. Auth/API rate limits are provider-managed and not verified in repository configuration.

11. Transactions and concurrency

No multi-statement application transaction is defined. Bookmark uniqueness plus upsert prevents duplicates for `(user, kind, reference)`. UI read-modify-write operations can race across devices; last accepted write wins. Tasbeeh session inserts are append-only. No idempotency key is used.

12. Performance and data risks

- Primary and bookmark unique indexes support principal lookup patterns.
- Consider an explicit index on `tasbeeh_sessions(user_id, created_at desc)` before large growth.
- Consider `bookmarks(user_id, kind, created_at desc)` if queries expand beyond the unique index order.
- JSON payload/image size is the main storage-exhaustion risk.
- No N+1 server query exists; multiple direct provider calls can still increase client latency.
- Full Hadith collections can be large; provider pagination/virtualized rendering should be assessed.

13. Sensitive fields never to return/log

Passwords, password hashes, refresh/access tokens, service-role keys, OAuth/SMTP secrets, raw database URLs, and unrelated Auth metadata. Client views should use only user ID/name/email needed for profile/account behavior. Current application does not receive password hashes.

14. Known gaps

Remote RLS state, Auth rate limits, CAPTCHA, token lifetime, refresh reuse protection, SMTP, Google settings, TLS database policy, backup/retention, and production indexes are Not verified from repository. Owner verification is required; see the security audit.

Implementation Plan

As Built

Product: Sakinah | Status: Current-State Roadmap | Version: 1.0 | Date: 2026-09-07

1. Current product status

Sakinah is deployed as a functional mobile-first PWA with guest use, Supabase authentication/owner data, Quran/Hadith integrations, prayer/Qibla utilities, Tasbeeh, guidance, More tools, dark mode, install onboarding, offline fallback, and versioned updates. Security hardening is committed in 296fcac and documented in docs/security/SECURITY-AUDIT.md.

2. Status inventory

Completed

- Core PWA shell, install manifest, standalone onboarding, service worker/update flow.
- Bottom navigation, sidebar, responsive shell, light/dark mode.
- Supabase Auth client and RLS schema.
- Quran Surah/Juz reader, translation/audio preferences, favorites/bookmarks.
- Quran extras: Seerah structure, recitation, subjects, 38 Fahm lessons, line modes.
- Eight Hadith collection metadata and remote collection/topic workflow.
- Live prayer request/countdown and Qibla bearing/sensor integration.
- Preset/custom Tasbeeh and history.
- Duas, Ibadat, Flashes/Greetings, Shahadat, Names, Waqf.
- Security headers, validation, privacy cache fix, security tests.

Partially implemented

- Books: catalog exists; full licensed downloadable books are not verified.
- Hadith translations: availability depends on provider editions.
- Seerah: seven-season UI indexes a fixed external playlist.
- Notifications: browser-local scheduling, not reliable server push.
- Profile: basic user/saved summary rather than full preference management.
- Offline: shell/images/selected Quran cache, not complete corpus download.

Missing or unverified

- Admin/CMS, account deletion/export, server push scheduler, full book storage, analytics/observability, remote Supabase security evidence, end-to-end browser tests, lint/type-check, accessibility audit.

3. Delivery governance

1. Work one phase at a time.
2. Do not start the next phase until tests pass and the phase is committed.
3. Update PRD/TRD before implementing changed requirements.
4. Preserve working code outside the phase.

5. Obtain owner confirmation for content licensing, provider substitution, schema semantics, auth behavior, or architecture changes.

4. Phase 0 - Security owner verification

Goal: Close controls that cannot be proved from source.

Current status: Required owner action.

Tasks

- Verify remote RLS with two dedicated test accounts.
- Confirm Supabase Site URL/redirect allow-list, Google callback, CAPTCHA, auth rate limits, token lifetime/reuse protection, and SMTP settings.
- Confirm Vercel variables contain only public browser values.
- Check deployed CSP without weakening it.

Dependencies: Owner access to Supabase, Google Cloud, Vercel; non-production test users.

Likely files: Documentation only unless a verified mismatch requires an approved patch.

Risks: Production auth interruption from incorrect redirects/policies.

Tests: Cross-owner negative read/write; signup/login/reset/OAuth; security-header/CSP browser test.

Done when: Evidence is attached to an owner-controlled audit record and all negative authorization tests pass.

Do not build/change yet: Database policies or Auth settings without backups and exact current-state capture.

5. Phase 1 - Quality baseline

Goal: Add trustworthy regression coverage without redesign.

Status: Planned.

Tasks: Select one existing-compatible browser test tool; cover guest launch, navigation, auth mock boundaries, Quran provider failure, prayer permission denial, Tasbeeh validation, PWA manifest; add lint/type-check only after owner approves conventions.

Dependencies: Phase 0; test accounts/API fixtures.

Files: `tests/`, `package.json`, possible non-production test config.

Risks: Brittle live-provider tests. Use fixtures/mocks for CI.

Verification: `npm run test:security`, future E2E command, `npm run build`, `npm audit --omit=dev`.

Done when: Critical navigation and owner-data paths have deterministic positive/negative tests.

Do not build/change yet: Product features or UI redesign.

6. Phase 2 - Toolchain security upgrade

Goal: Remove Vite/esbuild development advisories.

Status: Planned; owner approval required for major versions.

Tasks: Pin supported Node version; create upgrade branch; update Vite/React plugin compatibly; compare output, PWA, CSS, OAuth callback, and local mobile workflow; regenerate lockfile only as part of the approved upgrade.

Dependencies: Phase 1 coverage.

Files: `package.json`, `package-lock.json`, `vite.config.js`, CI/runtime config.

Risks: Node engine incompatibility and build behavior changes.

Done when: Full suite/build pass and `npm audit` has no applicable advisory.

Do not build/change yet: React major, Supabase provider, or hosting architecture.

7. Phase 3 - Data lifecycle and quota hardening

Goal: Bound user-owned storage and provide privacy lifecycle.

Status: Proposed, not approved.

Tasks: Measure existing payloads; design non-breaking DB constraints/indexes; define account deletion/export requirements; decide whether custom images move to private Supabase Storage; add migrations and rollback plan.

Dependencies: Owner product decisions, database backup, Phase 0 evidence.

Files: future migrations, data services, PRD/TRD first.

Risks: Existing oversized rows, data loss, storage cost.

Tests: migration against sanitized copy; quota boundaries; cascade/delete/export; RLS.

Done when: Approved retention policy and migration pass without production data loss.

Do not build/change yet: Do not alter `schema.sql` or production data before assessment.

8. Phase 4 - Content completeness and licensing

Goal: Establish a verified, licensed corpus contract.

Status: Partially implemented/unverified.

Tasks: Inventory each Quran translation, reciter, Hadith edition, Dua, book, image, font, and Seerah source; record license/attribution; define completeness checks; replace broken content only with owner/content-review approval.

Dependencies: Qualified Islamic content reviewer and licensing review.

Files: content adapters/assets and documentation after approval.

Risks: Incorrect religious text, broken attribution, copyright issues.

Tests: counts/checksums/provider schema, Arabic/Urdu glyph review, broken-link test.

Done when: Each corpus has provenance, license, expected count, and reviewer sign-off.

Do not build/change yet: Do not describe partial book/translation sets as complete.

9. Phase 5 - Reliability and performance

Goal: Improve measured load behavior and offline consistency.

Status: Existing foundations; further work planned.

Tasks: Capture Web Vitals on target Android/iOS; split the large App bundle further; virtualize large Hadith/Quran lists; add cache TTL/size policy; test update lifecycle and offline fallback; preserve coordinate privacy.

Dependencies: Phase 1 tests and content-provider contract.

Files: component boundaries, service worker, Vite config.

Risks: stale data, broken lazy routes, platform-specific PWA behavior.

Tests: cold/warm load, five reloads, offline/online transition, installed update, storage quota.

Done when: Owner-approved performance budgets pass on representative devices.

Do not build/change yet: Do not add aggressive caching for Auth, Supabase, or coordinate URLs.

10. Phase 6 - Accessibility and UI consistency

Goal: Make the existing design system more usable without rebranding.

Status: Planned.

Tasks: Standardize focus-visible states; audit contrast and 8–10 px labels; import/verify Arabic font; review no-selection policy; complete labels/roles; extend reduced-motion handling; test RTL content.

Dependencies: Accessibility acceptance criteria in PRD.

Files: CSS and components only after doc approval.

Risks: subtle layout changes; scripture typography regression.

Tests: keyboard-only, screen reader, zoom 200%, contrast, reduced motion, RTL screenshots.

Done when: Agreed WCAG target passes and branding remains intact.

Do not build/change yet: No palette/logo/navigation replacement.

11. Phase 7 - Notification architecture decision

Goal: Align reminder promises with browser platform limits.

Status: Decision required.

Tasks: Document current local reminder limitations; decide between local-only and true Web Push; if push is approved, define consent, subscription storage, scheduler, VAPID secret handling, timezone, unsubscribe, and privacy requirements before implementation.

Dependencies: Owner scope, privacy policy, backend architecture decision.

Risks: notification spam, secret handling, battery/platform restrictions.

Done when: Product wording matches tested delivery guarantees.

Do not build/change yet: Do not claim automatic Azan/background reliability without device evidence.

12. Testing and documentation gaps

- No automated browser E2E or component test suite.
- No lint/type-check command.
- No remote RLS test artifacts.
- No API schema contract tests or provider fixtures.
- No documented recovery/runbook or privacy policy.
- No formal content provenance register.

13. Deployment-readiness checklist

- Security owner verification complete.
- Tests/build/audit pass from clean checkout.
- Vercel headers/CSP verified.
- Supabase variables and redirects reviewed.
- PWA install/update tested on actual Android and iPhone.
- Provider failure/offline states tested.
- Content/license register approved.
- Rollback commit/deployment identified.

14. Traceable verification commands

```
npm install
npm run test:security
npm run build
npm audit --omit=dev
git diff --check
git status --short --branch
```

Every implementation phase must list its changed files, test output, unresolved risks, and commit before the next begins.

Traceability appendix

Product feature	PRD requirement	UI screen	API/operation	Entity	Role	Security control	Test evidence	Status
Account	PRD-FR-08/09	Sidebar/Auth/Profile	Supabase Auth	auth.users/profiles	Guest/User	PKCE, origin check, RLS	security tests + build	Implemented
Quran	PRD-FR-04	Quran/readers	Al Quran Cloud	bookmarks	All/User	bounds, CSP, RLS	build/manual provider needed	Implemented
Hadith	PRD-FR-05	Collection/topic/list	jsDelivr Hadith	bookmarks	All/User	path allow-list, RLS	build	Partial by edition
Prayer	PRD-FR-06	Prayer/banner	AlAdhan timings	none	All	coordinate bounds, no SW cache	security test	Implemented
Qibla	PRD-FR-06	Qibla	local calculation	none	All	permission boundary	build/device test needed	Implemented
Tasbeeh	PRD-FR-07	Tasbeeh	Supabase SDK	bookmarks/sessions	All/User	file/text bounds, RLS	security tests	Implemented
PWA	PRD-FR-01/10	Install/onboarding/offline	Service worker	Cache Storage	All	CSP, versioned cache	security tests/build	Implemented
Books	Current scope	Library	repository content	bookmark kind book	All/User	React escaping/RLS	content review needed	Partial

Key source evidence: `src/App.jsx`, `src/components/`, `src/lib/`, `supabase/schema.sql`, `public/sw.js`, `vercel.json`, `tests/security.test.js`, and `docs/security/SECURITY-AUDIT.md`.